

datalab 报告

姓名：张宇生 学号：2025201709

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse
37.00	1.00	1.00	2.00	4.00	4.00	3.00

logicalShift	leftBitCount	float_i2f	floatScale2	float64_f2i	floatPower2
3.00	4.00	4.00	4.00	3.00	4.00

```
chesed@LAPTOP-T6G22M2N:/mnt/c/Users/chesed/Desktop/datalab2026$ cd /mnt/c/Users/chesed/Desktop/datalab2026
26
make && python3 test.py
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
rm -f *.o btest fshow ishow *~ yacc.tab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd      1/1:    PASS
bitXor      1/1:    PASS
samesign    2/2:    PASS
logtwo      4/4:    PASS
byteSwap    4/4:    PASS
reverse     3/3:    PASS
logicalShift 3/3:    PASS
leftBitCount 4/4:    PASS
float_i2f    4/4:    PASS
floatScale2  4/4:    PASS
float64_f2i  3/3:    PASS
floatPower2  4/4:    PASS
Total points: 37
```

test 截图：test.py 全部通过，Total points: 37

1 解题报告

1.1 亮点

下面几道题写的比较久，写一下。有部分内容用了 AI。bitXor 是上限卡得最死的一道；logtwo 是合法符号少得反常，常规写法根本用不出来；float_i2f 的舍入我调了比较久；float64_f2i 上限给了 60，我只用了 16，是收得比较干净的一道。

1.2 bitXor

合法符号只有 ~ 和 &，上限 7 个。

我第一遍是把 $x \oplus y$ 拆成两个半边：

$$x \oplus y = \sim(\sim(x \& \sim y) \& \sim(\sim x \& y))$$

逻辑没问题，btest 也过了，但数了一遍 op 是 8 个，超了一个。后来换了个角度，从集合上看： $x \& y$ 是交集， $x \mid y$ 是并集，而“两位不一样”其实就是并集里挖掉交集，也就是

$$x \wedge y = (x \mid y) \& \sim(x \& y)$$

再把 $\mid$ 用德摩根换成 $\sim$ 和 $\&$ ，正好 7 个。

两者差的这一个 op 在哪，我想了一下：对称写法里两个半边各要多取一次反，而这一版 $\sim(x \& y)$ 的反是本来就要算的，省的就是它。所以题目上限写 7 应该就是卡在这两条路的差额上，多做一次取反就不够用了。

1.3 logtwo

合法符号是 $> < >> << \mid$ 。没有 $\&$ ，没有 $!$ ，没有 $+$ ，也没有 if 。

只能换问法。反正比较是允许的，那就别问“第几位是 1”，改成问“这个数还大于某个阈值吗”。比如先看 v 有没有大于 $0xFFFF$ ，大于的话答案至少是 16，把 v 右移 16 再往下问；不满足就留着看更小的位。依次试 16、8、4、2、1 五个步长就能覆盖 32 位。

中间改过一次。最早写的是 `int t = v & 0xFFFF;`，想先把低半段取出来再判断，写完才想起来 $\&$ 不在合法符号表里。用 `ai` 改成直接拿阈值常量比较之后，反而从 23 个降到 18 个，因为中间的变量和移位都省掉了。

1.4 float_i2f

把整数按位转成单精度浮点。这题麻烦的不是取位，是舍入。

流程是：先特判 0，然后拆出符号、对绝对值循环左移，把最高位的 1 顶到 bit31，移了多少次就记下来当指数用。剩下从高往低取 23 位当尾数，再往下的 8 位是要丢掉的部分。

舍入用 round-to-even，丢掉的那 8 位记作 `rnd`：

- `rnd` 大于 $0x80$ ，超过一半，直接进位；
- `rnd` 等于 $0x80$ 时刚好是一半，看尾数最后一位，是奇数才进位，这样舍入完结果一定是偶数；
- 小于 $0x80$ 就正常舍掉。

还有个很容易漏的地方：尾数加 1 之后有可能变成 $0x800000$ ，这就越过隐含位了，得把尾数清回 0、指数减 1。这一步不写的话绝大多数输入还是对的，只有少数几个会错，随手测不太容易撞上。

测的时候我挑了 $0x01000001$ 和 $0x01000003$ ，这两个正好卡在中间，一个该舍一个该进。另外 `INT_MIN` 也要注意，取绝对值那一步容易出问题。

2 反馈/收获/感悟/总结

全部写完花了大约 2 个小时。logtwo 和 float_i2f 思考的比较久，最后用了 ai 给解题思路我自己补充代码，其他难度都还好。

3 参考的重要资料

使用 deepseek v4.1 Flash 修改报告和提供部分解题思路